University of Piraeus
School of Information and Telecommunications Technologies
Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 1

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
23/10/2025

Abstract

The exercise analyzes the CVE-2014-0160 vulnerability (Heartbleed) and practically verifies the memory leak in a controlled environment (Docker Compose on an isolated Kali VM). Nmap and Metasploit were used for detection and confirmation.

Contents

Introduction to CVE-2014-0160 Vulnerability (Heartbleed Bug)

CVE-2014-0160, or commonly known as the Heartbleed Bug, is a vulnerability. CVE (Common Vulnerabilities and Exposures) is a software vulnerability registration and identification system, which assigns each vulnerability a unique identifier, so that it is easily detectable and recognizable internationally (NVD – CVE-2014-0160, n.d.). The number indicates the specific case and the year it was discovered (2014). This vulnerability created security issues in the OpenSSL cryptographic software library, allowing a malicious user to arbitrarily read portions of a server's memory, exposing sensitive data such as passwords, cookies, and private cryptographic keys (Simkin, 2014; Wikipedia contributors, 2025).

The OpenSSL library and SSL/TSL protocols

The OpenSSL library implements the TLS (Transport Layer Security) and SSL (Secure Sockets Layer) protocols, which ensure the exchange of data over the Internet (Cloudflare, n.d.). A website that implements the SSL/TLS protocols displays "HTTPS" in its address, instead of "HTTP", indicating that the communication is encrypted and secure (France, n.d.). The SSL protocol was designed in 1995 by Netscape to provide authentication and integrity (integrity) when transmitting sensitive data over the internet (Cloudflare, n.d.). In 1999, the IETF introduced the TLS protocol as a

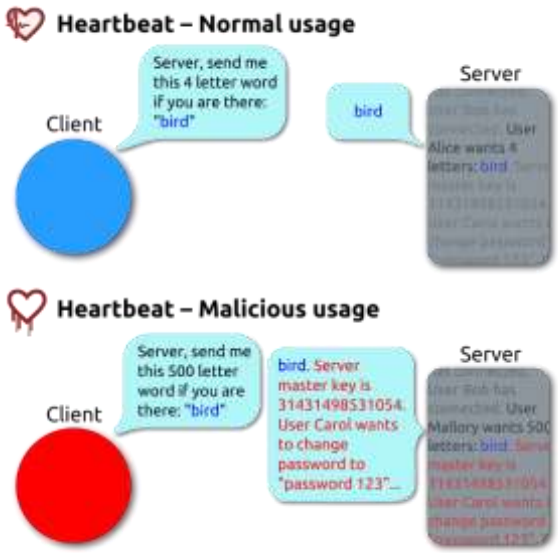
revamped and more secure version of SSL. Today, the two protocols are often referred to together as SSL/TLS, as they serve the same purpose. Over time, SSL has been completely replaced by TLS, which offers superior security and reliability (France, n.d.; The Open Group Blog, 2014). Nevertheless, the term 'SSL' is still widely used for historical reasons and because of the recognition it has gained since the early versions of secure internet connections.

Discovering and fixing Heartbleed

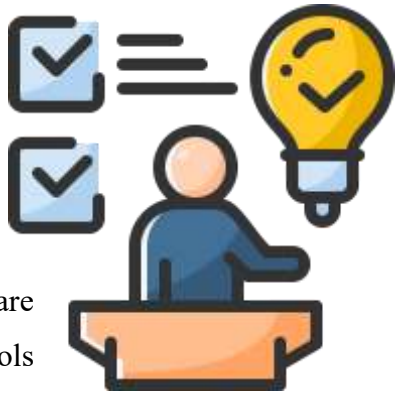
The Heartbleed bug was introduced into the source code of the OpenSSL library in December 2011, during the development of version 1.0.1, which was officially released on March 14, 2012 (Wikipedia contributors, 2025). The vulnerability went unnoticed for nearly two years, affecting millions of servers and devices running OpenSSL versions 1.0.1 to 1.0.1f (NVD – CVE-2014-0160, n.d.). In April 2014, the bug was independently discovered by two research teams, Google Security's Neel Mehta and Finnish cybersecurity firm Codenomicon. Although the Google team was the first to inform the developers of OpenSSL, both researchers detected it almost simultaneously (Balaganski, 2015). On April 7, 2014, the patched OpenSSL version 1.0.1g was released, which addressed the vulnerability (Simkin, 2014; Mandiant, 2014).

Technical description and effects of Heartbleed

The length of time the bug remained active proved decisive, as it had already left a strong imprint on internet security (Raywood, 2024). Heartbleed had a huge impact, as it allowed an attacker to read arbitrary portions of a server's memory using vulnerable versions of the OpenSSL library (Simkin, 2014; Wikipedia contributors, 2025). In particular, it took advantage of the Heartbeat extension of the TLS protocol, which allows one side of the connection to check whether the other side remains active by requesting the return of a small block of data (Balaganski, 2015). Due to a lack of control over the OpenSSL library, an attacker could falsely declare a longer length of data than was sent, forcing the server to return additional bytes of its memory (Mandiant, 2014; The Open Group Blog, 2014). The incident affected a large number of popular services (e.g., Yahoo, Tumblr, Dropbox, Google, Facebook, Intuit, Netflix) (Simkin, 2014; Google, 2014). As a result, organizations around the world have had to update their servers, reissue private keys and digital certificates, and ask users to change passwords, highlighting how destructive a simple logical imperfection in the code can be (Balaganski, 2015; Raywood, 2024). In this way, sensitive data such as usernames, passwords, cookies, authentication tokens, credit card numbers, and even private keys could be leaked (Simkin, 2014). If a private key is exposed, an attacker can impersonate a legitimate server, intercept encrypted communications, or carry out man-in-the-middle attacks on encrypted connections (France, n.d.; Cloudflare, n.d.). After the vulnerability was discovered, millions of organizations had to renew their digital certificates, issue new private keys, update their servers, and ask users to change passwords (The Open Group Blog, 2014; Balaganski, 2015). The incident highlighted that even a small bug in a few lines of open source can cause a global crisis of trust and affect the security of millions of users (Raywood, 2024; Timalsina, 2025).



The Importance and Impact of the Heartbleed Vulnerability



The Heartbleed vulnerability is not just limited to its immediate technical consequences—i.e., the leakage of data and private keys—but has become a milestone in the way we perceive online security today. Its discovery revealed how much the global web infrastructure depends on open source projects, which are often maintained with limited resources and controls (Raywood, 2024; Timalsina, 2025). The event brought into focus the need for sustainable support, funding and oversight of critical software projects and led to initiatives such as the establishment of the Core Infrastructure Initiative in April 2014 with the aim of all of the above (Linux Foundation, 2022). Additionally, he emphasized the importance of regularly updating software, renewing digital certificates and private keys after leaks, and being transparent and preparedness in responding to security incidents. In this way, it was realized that the protection of online infrastructure is not only a technical issue but includes organizational, social and economic elements and that another "simple" mistake in the code can disrupt the entire internet ecosystem (Balaganski, 2015; The Open Group Blog, 2014). Thus, Heartbleed is now claimed as a historical benchmark for cybersecurity.

Practical piece of work

Introduction:

We will start by opening a Virtual Machine (VMware) and install the latest version of Kali Linux (kali-linux-2025.3-vmware-amd64) there. Inside Kali Linux we will enable Docker and upload, with the help of Docker Compose, a controlled, vulnerable instance. We choose to run it inside the VM for isolation purposes and to be able to do a snapshot. This is how we protect the host and replicate vulnerable instances securely in an isolated environment, rather than running them directly on the host.

A snapshot is an imprint of the VM's state at a specific time. In other words, it includes the state of the operating system before the exploit was executed and allows us, after testing, to restore the VM to its original clean state without reinstalling.

In addition, Docker is the technology that runs containers, while Docker Compose is an organization tool that describes and launches multiple containers together through a docker-compose.yml file, making it easier to create the entire experimental environment. In other words, it saves us time to open many containers separately. With the above file they are activated at the same time. Without docker enabled, docker compose cannot work.

Vulnerable instance means a container that intentionally has an old, vulnerable version of the software (in this OpenSSL exercise with Heartbleed) so that we can use it as a "victim" in the VM.

Initiating Procedures:

We enter the terminal and install docker and docker compose using the following commands:

sudo apt install -y docker.io | For Docker

sudo apt-get install docker-compose-plugin | For Docker Compose

We update the list of available package versions with the following command:

sudo apt update

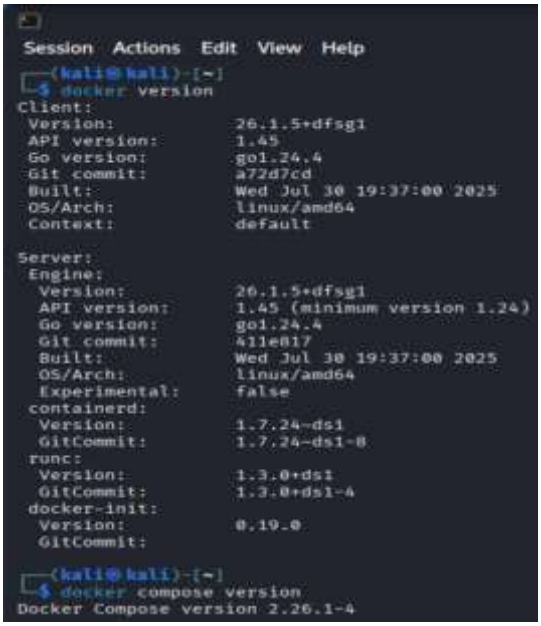
To update the applications we use the following command: **sudo apt upgrade**

After installing and updating all the necessary tools, a problem occurred with the mouse cursor in Kali Linux. The problem was solved by the following video: https://www.youtube.com/watch?v=r10w1Tn9-Lw

Then to make sure that docker and docker compose are "downloaded", we use the following commands:

Docker version | For Docker

Docker compose version | For Docker Compose



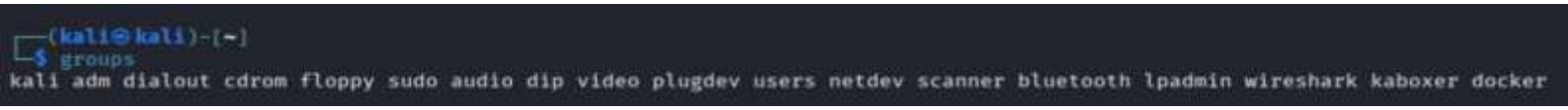
Then we use the following command to activate the background to create container:
sudo systemctl enable docker –now | activates docker for as long as the system is open (now) and will activate it every time we open the system. The above command is essentially a combination of the two:
sudo systemctl start docker | Docker is only starting for now

sudo systemctl enable docker | Booting Docker on Every Boot

In order to be able to run commands inside docker, we need to give the permissions to the current user. Normally the only user allowed to run commands inside docker is the administrator (root). The command for this is:

sudo usermod -aG docker $USER

The current user's access to docker is also confirmed by the "**groups**" command that shows what the user has access to, as shown in the image below:



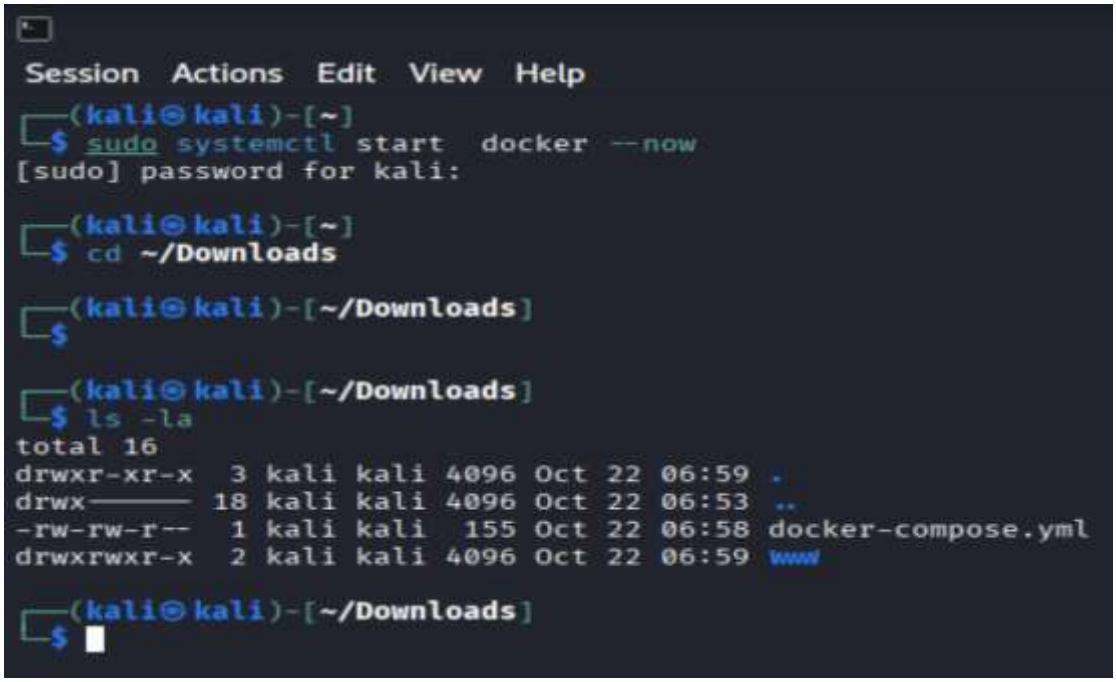
So from now on we can write commands in the terminal without "sudo". At this point we will take the first snapshot because our system is "clean", without having docker enabled, yet (we will return to this once the exercise is complete).

We proceed to activate docker with the following command:

systemctl start docker

Then we will open the link that includes the docker-cosmpose.yml and the www folder with the browser. The specific files will be placed in the Downloads folder. After the above process is done, we return to the terminal and go to the "Downloads" folder with the command:
cd ~/Downloads.

Then, with the command "**ls -la**", the files we downloaded will appear in the terminal.



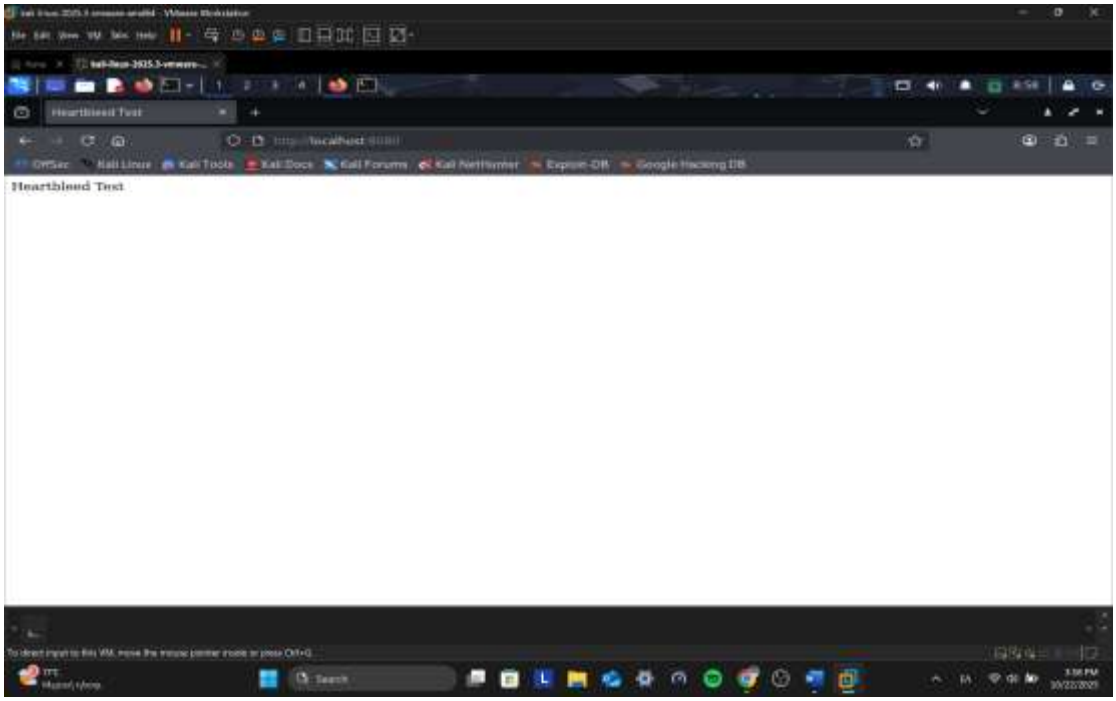
To make the container in this folder (Downloads) - which will "run" in the background - I will use the command:

docker compose up -d

It will essentially create the corresponding container based on docker-compose.yml file. Then with the command "**docker ps**" I will see the active containers as shown in the image below:

We observe that ports **80** and **443** correspond to the **HTTP** and **HTTPS** protocols respectively. Port 80 is used for simple, unencrypted communication over HTTP, while port 443 denotes a secure connection over HTTPS, which protects data with SSL/TLS encryption.

Then we open the browser and type http://localhost:8080 address as shown in the image below:



Since we see the index.html page, **we have successfully completed the instance configuration**. The next step before exploiting is **enumeration**, i.e. to scan the ports and check for vulnerabilities (eg. Heartbleed for this exercise). Before we start, we notice from the data of the last command we wrote in the terminal (docker ps) that the container is running the image with the vulnerable version of OpenSSL (1.0.1c) and as we mentioned above, the ports. Due to the theoretical analysis that preceded for the HTTP and HTTPS protocols, we assume that the port where Heartbleed may exist is 443 (HTTPS). To confirm this theory we will write the following command in the terminal:

sudo nmap -sV --script ssl-heartbleed -p 8443 localhost

```
┌──(kali㉿kali)-[~/Downloads]
└─$ sudo nmap -sV --script ssl-heartbleed -p 8443 localhost
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-22 09:18 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00046s latency).
Other addresses for localhost (not scanned): ::1

PORT     STATE SERVICE  VERSION
8443/tcp open  ssl/http nginx 1.11.13
|_http-server-header: nginx/1.11.13
| ssl-heartbleed:
|   VULNERABLE:
|   The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. It allows for stealing information intended to be
protected by SSL/TLS encryption.
|     State: VULNERABLE
|     Risk factor: High
|       OpenSSL versions 1.0.1 and 1.0.2-beta releases (including 1.0.1f and 1.0.2-beta1) of OpenSSL are affected by the Heartbleed bug. The bug allows for
 reading memory of systems protected by the vulnerable OpenSSL versions and could allow for disclosure of otherwise encrypted confidential information as w
ell as the encryption keys themselves.
|
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160
|       http://cvedetails.com/cve/2014-0160/
|_      http://www.openssl.org/news/secadv_20140407.txt

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.76 seconds
```

So through the **Nmap** tool we check the HTTPS port (8443) and prove that the port is vulnerable and that the web server is a vulnerable version of the OpenSSL library (Heartbleed/CVE-2014-0160).

We will now proceed from detecting to verifying the vulnerability using the **Metasploit tool**. In other words, we will exploit the vulnerability with this tool. We start with the command:

sudo msfconsole | to activate this tool

search heartbleed | searches the Metasploit library for this vulnerability

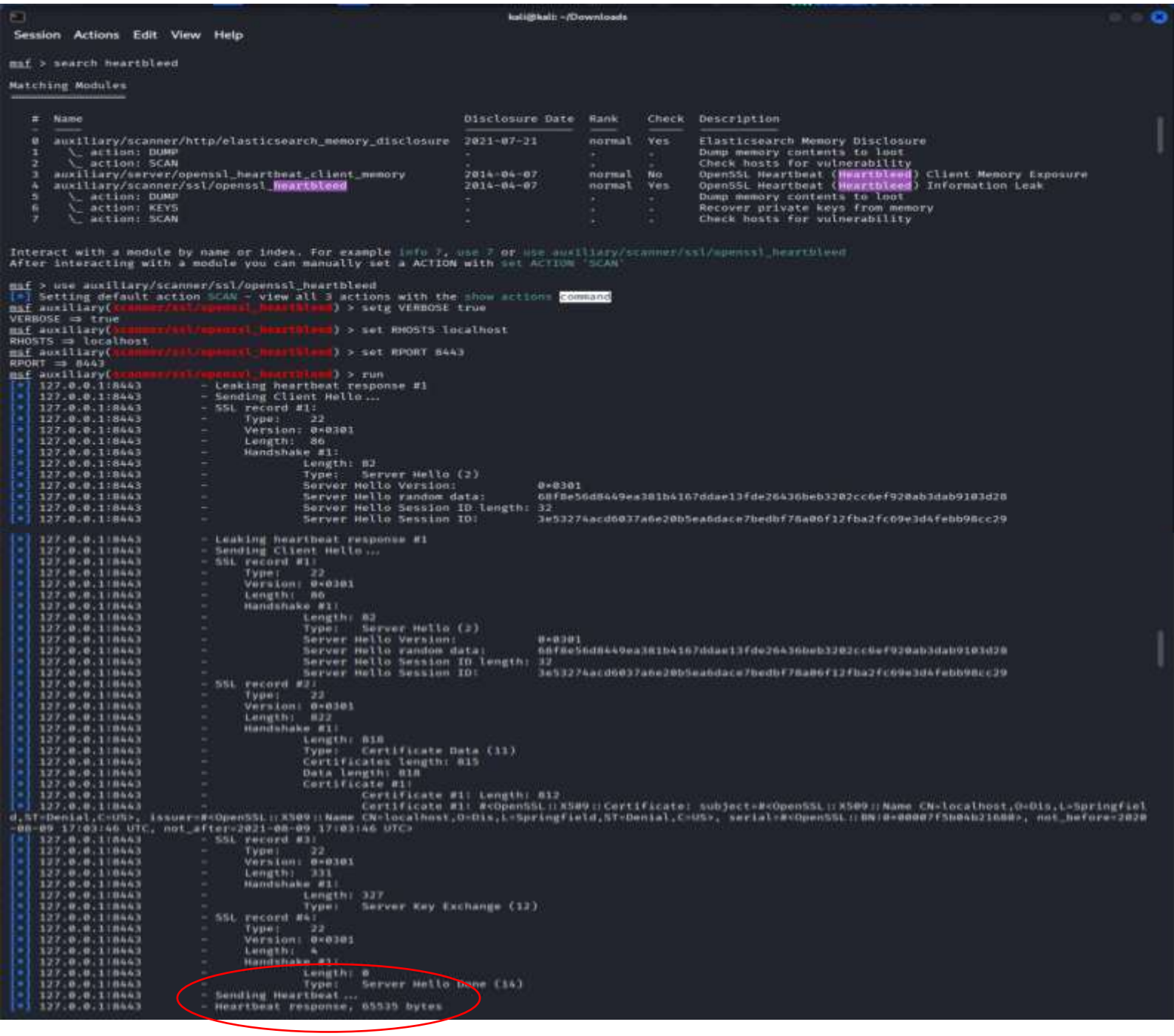
use auxiliary/scanner/ssl/openssl_heartbleed | Loads this module and checks if an SSL/TLS server is vulnerable to Heartbleed

setg VERBOSE true | By making the parameter true we observe a more detailed report from the module

set RHOSTS localhost | We set the module which target to scan

set RPORT 8443 | The module will target this port that we discovered has a vulnerability

run | We "run" the module with the parameters we set



What we need to keep from the image above is the command that sent a Heartbeat request and received a response of 65535 bytes. This is an indication of a successful memory leak. That is, the module was able to collect parts of the process's memory. Therefore the server is indeed vulnerable and there is a real risk of exposing sensitive information.

This snippet is part of the memory leak (approximately 65 KB) that the Heartbeat request returned. It may include sensitive elements from plaintext such as usernames, passwords, tokens, or even parts of private keys. It is therefore direct evidence of a memory leak from the vulnerability and must be treated as a serious incident.

Conclusion:

This exercise highlighted the process of identifying and verifying a vulnerability. The Nmap check identified an HTTPS port as vulnerable, while Metasploit confirmed the existence of an actual memory leak, proving that the web server can expose sensitive data that is running. The practical conclusion is that detection tools allow for the rapid identification of vulnerable systems and highlight the real risk of sensitive information leakage. Proper response includes immediately isolating or stopping the service, updating or replacing the vulnerable version of OpenSSL in this case, and checking for possible exposure of keys or credentials. Finally, it is critical to implement regular security audits, proper update management, and active monitoring through logs, so that

older or vulnerable versions of libraries such as OpenSSL can be detected and patched in a timely manner before they can be exploited by attackers.

Bibliography

Anna. (2025, June 29). Understanding CVE-2014-0160 (Heartbleed) [Video file]. Retrieved from https://www.youtube.com/watch?v=lj4J9Bgk5II

Balaganski, A. (2015, March 8). Lessons learned from the Heartbleed incident. Retrieved from https://www.kuppingercole.com/blog/balaganski/lessons-learned-from-the-heartbleed-incident

Blackduck, Inc. (2025, March 7). Heartbleed bug. Retrieved from https://www.heartbleed.com/

France, N. (n.d.). SSL Deprecation: Why TLS took over internet security. Retrieved from https://www.sectigo.com/resource-library/ssl-deprecation-understanding-the-evolution-of-security-protocols

Google. (2014, April 9). Google Services Updated to Address OpenSSL CVE-2014-0160 (the Heartbleed bug). Retrieved from https://security.googleblog.com/2014/04/google-services-updated-to-address.html

Lee, T. B. (2015, May 14). The Heartbleed Bug, explained. Vox. Retrieved from https://www.vox.com

Mandiant. (2014, April 18). Attackers exploit the heartbleed OpenSSL vulnerability to circumvent multi-factor authentication on VPNs. Retrieved from https://cloud.google.com/blog/topics/threat-intelligence/attackers-exploit-heartbleed-openssl-vulnerability/

Never Let a Good Crisis Go to Waste: Core Infrastructure Initiative - Linux Foundation. (2022, September 13). Retrieved from https://www.linuxfoundation.org/blog/blog/never-let-a-good-crisis-go-to-waste-core-infrastructure-initiative

NVD - CVE-2014-0160. (n.d.). Retrieved from https://nvd.nist.gov/vuln/detail/CVE-2014-0160

Raywood, D. (2024, April 29). Ten years of Heartbleed: Lessons learned. Retrieved from https://www.scworld.com/analysis/ten-years-of-heartbleed-lessons-learned

Simkin, S. (2014, April 11). Real-world Impact of Heartbleed (CVE-2014-0160): The Web is Just the Start. Retrieved from https://www.paloaltonetworks.com/blog/2014/04/real-world-impact-heartbleed-cve-2014-0160-web-just-start/

The Open Group Blog. (2014, May 8). Heartbleed: Tips and Lessons Learned. Retrieved from https://blog.opengroup.org/2014/04/24/heartbleed-tips-and-lessons-learned/

Timalsina, R., & Timalsina, R. (2025, July 9). The Heartbleed Bug: Lessons learned for system administrators. Retrieved from https://tuxcare.com/blog/heartbleed-bug/

What is SSL (Secure Sockets Layer)? | Cloudflare. (n.d.). Retrieved from https://www.cloudflare.com/learning/ssl/what-is-ssl/

Wikipedia contributors. (2025, October 3). Heartbleed. Retrieved from https://en.wikipedia.org/wiki/Heartbleed

On the practical part of the work:

Increase terminal scale/size on Kali Linux running in VMWare. (n.d.). Retrieved from https://unix.stackexchange.com/questions/700768/increase-terminal-scale-size-on-kali-linux-running-in-vmware

GeeksforGeeks. (2024, September 19). Detecting and Exploiting the OpenSSL Heartbleed Vulnerability with Nmap and Metasploit. Retrieved from https://www.geeksforgeeks.org/linux-unix/detect-exploit-heartbleed-vulnerability-nmap-metasploit/

Installing docker on Kali Linux | Kali Linux Documentation. (n.d.). Retrieved from https://www.kali.org/docs/containers/installing-docker-on-kali/

Self Host with IPv6rs - IPv6 Provider - How to Install Docker Compose on Kali Linux Latest. (n.d.). Retrieved from https://ipv6.rs/tutorial/Kali_Linux_Latest/Docker_Compose/

Russell Thackston. (2020, February 24). *[046] Creating a VM snapshot in VMWare* [Video file]. Retrieved from https://www.youtube.com/watch?v=U4WDkjm9D5A

WsCube Cyber Security. (2025, October 14). *How to fix invisible mouse cursor in Kali Linux (Permanent Solution)* [Video file]. Retrieved from https://www.youtube.com/watch?v=r10w1Tn9-Lw

GetCyber. (2024, February 1). *EASY! Install docker on kali Linux!* [Video file]. Retrieved from https://www.youtube.com/watch?v=exyXqfOD7MQ

Barve, A. (2025, February 11). What is Port 443? A Technical Guide for HTTPS Port 443. Retrieved from https://www.ssl2buy.com/wiki/port-443

What is Port 80? (n.d.). Retrieved from https://www.cbtnuggets.com/common-ports/what-is-port-80

Scribbr. (2025, February 17). Free Citation Generator | APA, MLA, Chicago | Scribbr. Retrieved from https://www.scribbr.com/citation/generator/folders/zMjr12dSZioRFnxU1CSeN/lists/2FEMpjRVM6Fagn84jvHoWP/

ChatGPT. (n.d.). Retrieved from https://chatgpt.com/